

llama.cpp

```
/*
notes: 杨小兵-2024-12-16

1、这个项目名称叫做llama.cpp
*/
```



```
/*
杨小兵-2024-12-16

1、这个markdown语法为了在渲染的时候展示一张项目的logo图片
2、其中[llama]为了给这个图片命名
3、这个logo图片是从互联网中获取得到的
*/
```

license

MIT

Server

passing

```
/*
杨小兵-2024-12-16

1、通过点击图片的方式在浏览器中打开对应的网址
2、这里涉及到两个网址：
    2.1 许可证
    2.2 项目服务器
*/
```

[Roadmap](#) / [Project status](#) / [Manifesto](#) / [ggml](#)

```
/*
杨小兵-2024-12-16

1、通过点击文字链接的方式跳转到该项目的不同网址查看对应的内容
    1.1 roadmap
    1.2 project status
    1.3 manifesto
    1.4 ggml
*/
```

Inference of Meta's [LLaMA](#) model (and others) in pure C/C++

```
/*
杨小兵-2024-12-16

1、只使用C/C++对Meta公司的LLaMA模型、其他公司的模型进行inference
    1.1 meta's model
    1.2 others model
2、这里的亮点就是只使用C、C++来完成整个模型的推理工作而不会使用到其他的高级语言，对于不同平台的适配将会变得容易
*/
```

1 Recent API changes

- [Changelog for libllama API](#)
- [Changelog for llama-server REST API](#)

```
/*
杨小兵-2024-12-16

1、这部分内容为了说明最近函数API的一些改变情况
2、通过日志的方式来原因libllama接口变化情况
    2.1 libllama api
3、通过日志的方式来原因llama-server接口变化情况
    3.1 llama-server rest api
*/
```

2 Hot topics

- **Introducing GGUF-my-LoRA** <https://github.com/ggerganov/llama.cpp/discussions/10123>
- Hugging Face Inference Endpoints now support GGUF out of the box!
<https://github.com/ggerganov/llama.cpp/discussions/9669>
- Hugging Face GGUF editor: [discussion](#) | [tool](#)

```
/*  
杨小兵-2024-12-16  
  
1、这部分内容为了说明一些热点话题  
2、首先通过网址的方式来介绍GGUF-my-LoRA的话题  
3、hugging face inference endpoints使得用户可以更方便地在 Hugging Face 上部署使用  
GGUF 格式的模型，无需进行格式转换或额外的配置步骤  
4、提供了工具：对GGUF文件进行编辑  
*/
```

3 Description

The main goal of `llama.cpp` is to enable LLM inference with minimal setup and state-of-the-art performance on a wide range of hardware - locally and in the cloud.

```
/*  
杨小兵-2024-12-16  
  
1、llama.cpp项目的主要目标就是为了使用最小的配置和先进的性能在很多硬件上实现LLM的  
inference任务  
2、可以在本地、云端部署模型  
*/
```

- Plain C/C++ implementation without any dependencies
- Apple silicon is a first-class citizen - optimized via ARM NEON, Accelerate and Metal frameworks
- AVX, AVX2, AVX512 and AMX support for x86 architectures
- 1.5-bit, 2-bit, 3-bit, 4-bit, 5-bit, 6-bit, and 8-bit integer quantization for faster inference and reduced memory use
- Custom CUDA kernels for running LLMs on NVIDIA GPUs (support for AMD GPUs via HIP and Moore Threads MTT GPUs via MUSA)
- Vulkan and SYCL backend support
- CPU+GPU hybrid inference to partially accelerate models larger than the total VRAM capacity

```
/*  
杨小兵-2024-12-16  
  
1、没有任何依赖的纯C/C++实现  
2、苹果的芯片是被优先考虑的，通过 ARM NEON、Accelerate 和 Metal 框架进行了优化  
3、对于x86架构，AVX, AVX2, AVX512 and AMX指令集是被支持的  
4、为了更快的推理任务、更好的内存需求1.5-bit, 2-bit, 3-bit, 4-bit, 5-bit, 6-bit, and  
8-bit整数量化是被支持的  
5、为了在NVIDIA GPUS上运行LLMs，自定义了一些CUDA内核（通过HIP支持AMD GPUS、通过MUSA支持  
Moore Threads MTT GPUS）  
6、支持Vulkan and SYCL backend  
7、CPU+GPU 混合推理，部分加速大于 VRAM 总容量的模型  
*/
```

The `llama.cpp` project is the main playground for developing new features for the `ggml` library.

```
/*  
杨小兵-2024-12-16  
  
1、llama.cpp 项目是开发 [ggml](https://github.com/ggerganov/ggml) 库新功能的主要平台。  
2、llama.cpp项目用到了ggml项目中的东西，llama项目利用ggml项目的计算能力实现对不同的LLMs推理工作  
*/
```

► Models

Typically finetunes of the base models below are supported as well.

Instructions for adding support for new models: [HOWTO-add-model.md](#)

```
/*  
杨小兵-2024-12-16  
  
1、这里提到的模型也包含一些基础模型的微调版本，这些微调版本的模型也是被llama.cpp项目支持的  
2、如果想要在llama.cpp项目中添加对新模型的支持，这里的HOWTO-add-model.md文件提供了参考  
*/
```

Text-only

- ☒ LLaMA 🐑
- ☒ LLaMA 2 🐑 🐑
- ☒ LLaMA 3 🐑 🐑 🐑
- ☒ Mistral 7B
- ☒ Mixtral MoE
- ☒ DBRX
- ☒ Falcon
- ☒ Chinese LLaMA / Alpaca and Chinese LLaMA-2 / Alpaca-2
- ☒ Vigogne (French)
- ☒ BERT
- ☒ Koala
- ☒ Baichuan 1 & 2 + derivations
- ☒ Aquila 1 & 2
- ☒ Starcoder models
- ☒ Refact
- ☒ MPT
- ☒ Bloom
- ☒ Yi models

- ☒ [StableLM models](#)
- ☒ [Deepseek models](#)
- ☒ [Qwen models](#)
- ☒ [PLaMo-13B](#)
- ☒ [Phi models](#)
- ☒ [GPT-2](#)
- ☒ [Orion 14B](#)
- ☒ [InternLM2](#)
- ☒ [CodeShell](#)
- ☒ [Gemma](#)
- ☒ [Mamba](#)
- ☒ [Grok-1](#)
- ☒ [Xverse](#)
- ☒ [Command-R models](#)
- ☒ [SEA-LION](#)
- ☒ [GritLM-7B + GritLM-8x7B](#)
- ☒ [OLMo](#)
- ☒ [OLMo 2](#)
- ☒ [OLMoE](#)
- ☒ [Granite models](#)
- ☒ [GPT-NeoX + Pythia](#)
- ☒ [Snowflake-Arctic MoE](#)
- ☒ [Smaug](#)
- ☒ [Poro 34B](#)
- ☒ [Bitnet b1.58 models](#)
- ☒ [Flan T5](#)
- ☒ [Open Elm models](#)
- ☒ [ChatGLM3-6b + ChatGLM4-9b](#)
- ☒ [SmolLM](#)
- ☒ [EXAONE-3.0-7.8B-Instruct](#)
- ☒ [FalconMamba Models](#)
- ☒ [Jais](#)
- ☒ [Bielik-11B-v2.3](#)
- ☒ [RWKV-6](#)
- ☒ [GigaChat-20B-A3B](#)

Multimodal

- ☒ [LLaVA 1.5 models, LLaVA 1.6 models](#)
- ☒ [BakLLaVA](#)
- ☒ [Obsidian](#)
- ☒ [ShareGPT4V](#)
- ☒ [MobileVLM 1.7B/3B models](#)
- ☒ [Yi-VL](#)
- ☒ [Mini CPM](#)
- ☒ [Moondream](#)
- ☒ [Bunny](#)

-  [Qwen2-VL](#)

► Bindings

- Python: [abetlen/llama-cpp-python](#)
- Go: [go-skynet/go-llama.cpp](#)
- Node.js: [withcatai/node-llama-cpp](#)
- JS/TS (llama.cpp server client): [lgrammel/modelfusion](#)
- JS/TS (Programmable Prompt Engine CLI): [offline-ai/cli](#)
- JavaScript/Wasm (works in browser): [tangledgroup/llama-cpp-wasm](#)
- Typescript/Wasm (nicer API, available on npm): [ngxson/wllama](#)
- Ruby: [yoshoku/llama_cpp.rb](#)
- Rust (more features): [edgenai/llama_cpp-rs](#)
- Rust (nicer API): [mdrokz/rust-llama.cpp](#)
- Rust (more direct bindings): [utilityai/llama-cpp-rs](#)
- C#/.NET: [SciSharp/LLamaSharp](#)
- C#/VB.NET (more features - community license): [LM-Kit.NET](#)
- Scala 3: [donderom/llm4s](#)
- Clojure: [phronmophobic/llama.clj](#)
- React Native: [mybigday/llama.rn](#)
- Java: [kherud/java-llama.cpp](#)
- Zig: [deins/llama.cpp.zig](#)
- Flutter/Dart: [netdur/llama_cpp_dart](#)
- Flutter: [xuegao-tzx/Fllama](#)
- PHP (API bindings and features built on top of llama.cpp): [distantmagic/resonance](#) ([more info](#))
- Guile Scheme: [guile_llama_cpp](#)
- Swift [srgtuszy/llama-cpp-swift](#)
- Swift [ShenghaiWang/SwiftLlama](#)

► UIs

(to have a project listed here, it should clearly state that it depends on [LLama.cpp](#))

- [AI Sublime Text plugin](#) (MIT)
- [cztomsik/ava](#) (MIT)
- [Dot](#) (GPL)
- [eva](#) (MIT)
- [iohub/collama](#) (Apache-2.0)
- [janhq/jan](#) (AGPL)
- [KanTV](#) (Apache-2.0)
- [KodiBot](#) (GPL)
- [llama.vim](#) (MIT)
- [LARS](#) (AGPL)
- [Llama Assistant](#) (GPL)
- [LLMFarm](#) (MIT)
- [LLMUnity](#) (MIT)
- [LMStudio](#) (proprietary)
- [LocalAI](#) (MIT)

- [LostRuins/koboldcpp](#) (AGPL)
 - [MindMac](#) (proprietary)
 - [MindWorkAI/AI-Studio](#) (FSL-1.1-MIT)
 - [Mobile-Artificial-Intelligence/maid](#) (MIT)
 - [Mozilla-Ocho/llamafire](#) (Apache-2.0)
 - [nat/openplayground](#) (MIT)
 - [nomic-ai/gpt4all](#) (MIT)
 - [ollama/ollama](#) (MIT)
 - [oobabooga/text-generation-webui](#) (AGPL)
 - [PocketPal AI](#) (MIT)
 - [psugihara/FreeChat](#) (MIT)
 - [ptsochantaris/emeltal](#) (MIT)
 - [pythops/tenere](#) (AGPL)
 - [ramalama](#) (MIT)
 - [semperai/amica](#) (MIT)
 - [withcatai/catai](#) (MIT)
- Tools
- [akx/ggify](#) – download PyTorch models from HuggingFace Hub and convert them to GGML
 - [akx/ollama-dl](#) – download models from the Ollama library to be used directly with llama.cpp
 - [crashr/gppm](#) – launch llama.cpp instances utilizing NVIDIA Tesla P40 or P100 GPUs with reduced idle power consumption
 - [gpustack/gguf-parser](#) - review/check the GGUF file and estimate the memory usage
 - [Styled Lines](#) (proprietary licensed, async wrapper of inference part for game development in Unity3d with pre-built Mobile and Web platform wrappers and a model example)
- Infrastructure
- [Paddler](#) - Stateful load balancer custom-tailored for llama.cpp
 - [GPUStack](#) - Manage GPU clusters for running LLMs
 - [llama_cpp_canister](#) - llama.cpp as a smart contract on the Internet Computer, using WebAssembly
- Games
- [Lucy's Labyrinth](#) - A simple maze game where agents controlled by an AI model will try to trick you.

4 Supported backends

Backend	Target devices
Metal	Apple Silicon
BLAS	All
BLIS	All
SYCL	Intel and Nvidia GPU
MUSA	Moore Threads MTT GPU

Backend	Target devices
CUDA	Nvidia GPU
hipBLAS	AMD GPU
Vulkan	GPU
CANN	Ascend NPU

```
/*
杨小兵-2024-12-16

1、Backend（后端）：在软件开发和计算框架中，**backend** 指的是处理底层计算任务的模块或库。它们负责执行核心计算、优化性能、管理硬件资源等。不同的 backend 通常针对不同的硬件架构或计算平台进行优化，以充分利用特定设备的计算能力和特性。
2、Target Devices（目标设备）：指的是应用程序或计算任务所运行的具体硬件设备。这些设备可以包括各种处理器（如 CPU、GPU）、专用加速器（如 NPU）、或其他计算单元。不同的目标设备具有不同的架构和性能特点，因此选择合适的 backend 可以确保软件在这些设备上高效运行。
3、backend and target devices
    3.1 Metal 是 Apple 提供的底层图形和计算 API，专为 Apple 的硬件（如 M1、M2 芯片）优化，能够充分利用 Apple Silicon 的图形和计算能力。
    3.2 BLAS 是一组用于执行基本线性代数运算的标准库，广泛应用于各种计算平台，适用于所有类型的硬件设备。
    3.3 BLIS 是一个高性能的线性代数库，旨在提供可扩展和可定制的 BLAS 实现，适用于各种硬件平台。
    3.4 SYCL 是一种基于 C++ 的异构计算框架，允许开发者编写跨平台代码，支持 Intel 和 Nvidia 的 GPU 设备，便于在不同硬件上进行并行计算。
    3.5 MUSA 是针对 Moore Threads 公司生产的 MTT GPU 设备优化的计算后端，旨在充分利用其 GPU 的并行计算能力。
    3.6 CUDA 是 Nvidia 提供的并行计算平台和编程模型，专为 Nvidia 的 GPU 设计，广泛应用于深度学习、科学计算等领域。
    3.7 hipBLAS 是 AMD 提供的 BLAS 实现，基于 HIP（Heterogeneous-Compute Interface for Portability）框架，专为 AMD GPU 优化，支持高性能线性代数运算。
    3.8 Vulkan 是一个跨平台的低开销图形和计算 API，支持多种 GPU 设备，提供高效的并行计算能力，适用于需要高性能图形和计算的应用。
    3.9 CANN 是华为开发的用于神经网络计算的架构，专为 Ascend 系列的神经处理单元（NPU）设计，优化了深度学习任务的计算性能。
*/
```

5 Building the project

The main product of this project is the **llama** library. Its C-style interface can be found in [include/llama.h](#). The project also includes many example programs and tools using the **llama** library. The examples range from simple, minimal code snippets to sophisticated sub-projects such as an OpenAI-compatible HTTP server. Possible methods for obtaining the binaries:

```
/*
杨小兵-2024-12-16
```


- 1、这部分内容讲述的是如何构建llama.cpp项目
 - 2、这个项目的主要产品成果就是llama库。这个项目的C风格接口可以在include/llama.h中找到
 - 3、这个项目包含了一些示例程序和工具
 - 3.1 example programs用到了llama library
 - 3.2 tools用到了llama library
 - 4、这些示例程序从简单的、最小化的代码片段到复杂的子项目例如与OpenAI兼容HTTP服务器。
 - 5、可以通过下列内容获取这些可执行二进制文件
- */

- Clone this repository and build locally, see [how to build](#)
- On MacOS or Linux, install **llama.cpp** via [brew](#), [flox](#) or [nix](#)
- Use a Docker image, see [documentation for Docker](#)
- Download pre-built binaries from [releases](#)

```
/*
杨小兵-2024-12-16

1、将这个仓库clone到本地并且在本地进行构建，查看docs/build.md文档获取帮助
2、在MacOS或者Linux上可以通过brew, flox or nix来对llama.cpp项目成果进行安装（不需要编译构建，只需要直接进行下载）
3、如果想要使用Docker镜像，那么可以查看docs/docker.md获取帮助
4、可以从https://github.com/ggerganov/llama.cpp/releases直接下载已经构建好的二进制文件
*/
```

6 Obtaining and quantizing models

The [Hugging Face](#) platform hosts a [number of LLMs](#) compatible with **llama.cpp**:

- [Trending](#)
- [LLaMA](#)

```
/*
杨小兵-2024-12-16

1、获取模型、量化模型
2、Hugging Face 平台拥有许多与 llama.cpp 兼容的LLMs
3、上述的两个链接为了在hugging face上查找GGUF相关的LLMs
*/
```

After downloading a model, use the CLI tools to run it locally - see below.

llama.cpp requires the model to be stored in the [GGUF](#) file format. Models in other data formats can be converted to GGUF using the [convert_*.py](#) Python scripts in this repo.

```
/*
杨小兵-2024-12-16
```

```
1、将模型加载之后使用CLI工具在本地运行模型
2、llama.cpp项目要求模型以GGUF文件格式进行保存，以其他数据格式保存的模型可以使用该仓库中的convert_*.py的python脚本进行转化
*/
```

The Hugging Face platform provides a variety of online tools for converting, quantizing and hosting models with `llama.cpp`:

```
/*
杨小兵-2024-12-16

1、Hugging Face平台提供了多种在线工具，用于使用`llama.cpp`转换，量化和托管模型
    1.1 转化模型
    1.2 量化模型
    1.3 托管模型
*/
```

- Use the [GGUF-my-repo space](#) to convert to GGUF format and quantize model weights to smaller sizes
- Use the [GGUF-my-LoRA space](#) to convert LoRA adapters to GGUF format (more info: <https://github.com/ggerganov/llama.cpp/discussions/10123>)
- Use the [GGUF-editor space](#) to edit GGUF meta data in the browser (more info: <https://github.com/ggerganov/llama.cpp/discussions/9268>)
- Use the [Inference Endpoints](#) to directly host `llama.cpp` in the cloud (more info: <https://github.com/ggerganov/llama.cpp/discussions/9669>)

```
/*
杨小兵-2024-12-16

1、使用GGUF-my-repo space对模型格式进行转化并且对模型进行量化从而减少模型存储大小
2、使用GGUF-my-LoRA space将LoRA adapters转化为GGUF格式，可以通过
https://github.com/ggerganov/llama.cpp/discussions/10123查看更多信息
3、使用GGUF-editor space在浏览器中来对GGUF文件的meta数据进行编辑
4、使用Inference Endpoints直接在云中部署llama.cpp项目
*/
```

To learn more about model quantization, [read this documentation](#)

```
/*
杨小兵-2024-12-16

1、查看examples/quantize/README.md文件来获取关于模型量化的更多内容
*/
```

7 llama-cli

A CLI tool for accessing and experimenting with most of `llama.cpp`'s functionality.

```
/*
杨小兵-2024-12-16

1、用于访问和试验“llama.cpp”大部分功能的 CLI 工具
2、llama-cli用来测试和实验llama.cpp项目的大部分功能
*/
```

- ▼ Run simple text completion

```
llama-cli -m model.gguf -p "I believe the meaning of life is" -n 128

# I believe the meaning of life is to find your own truth and to live in
accordance with it. For me, this means being true to myself and following my
passions, even if they don't align with societal expectations. I think
that's what I love about yoga - it's not just a physical practice, but a
spiritual one too. It's about connecting with yourself, listening to your
inner voice, and honoring your own unique journey.
```

- Run in conversation mode

```
llama-cli -m model.gguf -p "You are a helpful assistant" -cnv

# > hi, who are you?
# Hi there! I'm your helpful assistant! I'm an AI-powered chatbot designed
to assist and provide information to users like you. I'm here to help answer
your questions, provide guidance, and offer support on a wide range of
topics. I'm a friendly and knowledgeable AI, and I'm always happy to help
with anything you need. What's on your mind, and how can I assist you today?
#
# > what is 1+1?
# Easy peasy! The answer to 1+1 is... 2!
```

- Run with custom chat template

```
# use the "chatml" template
llama-cli -m model.gguf -p "You are a helpful assistant" -cnv --chat-
template chatml

# use a custom template
llama-cli -m model.gguf -p "You are a helpful assistant" -cnv --in-prefix
'User: ' --reverse-prompt 'User:'
```

[Supported templates](#)

- ► Constrain the output with a custom grammar

```
llama-cli -m model.gguf -n 256 --grammar-file grammars/json.gbnf -p
'Request: schedule a call at 8pm; Command:'

# {"appointmentTime": "8pm", "appointmentDetails": "schedule a a call"}
```

The [grammars/](#) folder contains a handful of sample grammars. To write your own, check out the [GBNF Guide](#).

For authoring more complex JSON grammars, check out <https://grammar.intrinsiclabs.ai/>

```
/*
杨小兵-2024-12-19

1、这一部分的内容需要进一步了解
2、grammar用于约束模型（如 LLaMA）生成的输出，使其符合预定义的结构和格式。这种约束
确保了输出数据的一致性和可解析性，特别是在需要将自然语言转换为结构化数据（如 JSON）
的场景中尤为重要。
3、通过 GBNF 文件，用户可以详细描述期望的输出结构，使模型生成的内容严格遵循这些规
则。
*/
```

8 llama-server

```
/*
杨小兵-2024-12-16

1、这部分用来介绍示例程序中的llama-server，这是个兼容OpenAI的HTTP server，这部分内容是
比较重要的
*/
```

A lightweight, [OpenAI API](#) compatible, HTTP server for serving LLMs.

```
/*
杨小兵-2024-12-16

1、一个轻量级的、openai api兼容的、为了运行LLMs的HTTP服务
2、特点
    2.1 轻量级
    2.2 兼容OpenAI API
    2.3 serving for LLMs
*/
```

- ▼ Start a local HTTP server with default configuration on port 8080

```
llama-server -m model.gguf --port 8080
```

```
# Basic web UI can be accessed via browser: http://localhost:8080
# Chat completion endpoint: http://localhost:8080/v1/chat/completions
```

```
/*
```

```
杨小兵-2024-12-19
```

1、命令解释

- 1.1 llama-server 二进制程序
- 1.2 -m model.gguf 模型文件
- 1.3 --port 8080 服务器端口

2、效果解释

- 1.1 可以通过浏览器网址：<http://localhost:8080>进行访问基本页面
- 1.2 对话补全端点地址：<http://localhost:8080/v1/chat/completions>

```
*/
```

- Support multiple-users and parallel decoding

```
# up to 4 concurrent requests, each with 4096 max context
llama-server -m model.gguf -c 16384 -np 4
```

```
/*
```

```
杨小兵-2024-12-19
```

1、支持多用户、并行解码

2、最多支持4个并发请求，每一个用户最多4096上下文长度

```
*/
```

- Enable speculative decoding

```
# the draft.gguf model should be a small variant of the target model.gguf
llama-server -m model.gguf -md draft.gguf
```

```
/*
```

```
杨小兵-2024-12-16
```

1、-m model.gguf 指定了主要使用的模型文件为 `model.gguf`

2、-md draft.gguf 指定了一个辅助模型文件 `draft.gguf`，用于推测性解码

3、`draft.gguf` 模型应该是目标模型 `model.gguf` 的一个小型变体。这意味着 `draft.gguf` 是一个轻量级或精简版的模型，用于辅助或优化目标模型的运行

4、注意：这里两个模型是必须要相同系列的

*/

- Serve an embedding model

```
# use the /embedding endpoint
llama-server -m model.gguf --embedding --pooling cls -ub 8192
```

```
/*
杨小兵-2024-12-16

1、可以作为一个embedding model
2、命令解释
    2.1 llama-server    可执行文件
    2.2 -m model.gguf   指定模型文件
    2.3 --embedding     限制仅支持嵌入用例
    2.4 --pooling cls   嵌入的池化类型，如果未指定则使用模型默认值
    2.5 -ub 8192        物理最大批次大小
*/
```

- Serve a reranking model

```
# use the /reranking endpoint
llama-server -m model.gguf --reranking
```

```
/*
杨小兵-2024-12-19

1、命令解释
    2.1 llama-server    可执行文件
    2.2 -m model.gguf   指定模型文件
    2.3 --reranking     在服务器上启用重新排名端点
*/
```

- Constrain all outputs with a grammar

```
# custom grammar
llama-server -m model.gguf --grammar-file grammar.gbnf

# JSON
llama-server -m model.gguf --grammar-file grammars/json.gbnf
```

```
/*
杨小兵-2024-12-19

1、使用grammer来限制模型的所有输出
2、命令解释
    2.1 llama-server                可执行文件
    2.2 -m model.gguf               指定模型文件
    2.3 --grammar-file grammar.gbnf  自定义grammar
    2.4 --grammar-file grammars/json.gbnf  JSON grammar
*/
```

9 llama-perplexity

A tool for measuring the perplexity ^1 (and other quality metrics) of a model over a given text.

```
/*
杨小兵-2024-12-16

1、`llama-perplexity` 是一个用于测量模型在给定文本上的困惑度 ( perplexity ) 以及其他质量指标的工具。困惑度是一种衡量语言模型在预测文本时的不确定性的方法。具体来说，它量化了模型对给定文本的预测能力：困惑度越低，意味着模型的预测越精确。
2、`llama-perplexity` 通过计算语言模型在处理某段文本时的困惑度，从而评估该模型在理解该文本时的效果。它是通过给定模型和文本文件，返回一组困惑度的数值，最终得出该文本的平均困惑度值。
*/
```

- ▼ Measure the perplexity over a text file

```
llama-perplexity -m model.gguf -f file.txt

# [1]15.2701,[2]5.4007,[3]5.3073,[4]6.2965,[5]5.8940,[6]5.6096,[7]5.7942,[8]4.9297, ...
# Final estimate: PPL = 5.4007 +/- 0.67339
```

- Measure KL divergence

```
# TODO
```

10 llama-bench

Benchmark the performance of the inference for various parameters.

```
/*
杨小兵-2024-12-16

1、针对多种不同参数模型推理性能的基准测试
2、llama-bench为了测量模型的性能
*/
```

- ▼ Run default benchmark

```
llama-bench -m model.gguf

# Output:
# | model | size | params | backend | threads |
# |-----|-----|-----|-----|-----|
# | qwen2 1.5B Q4_0 | 885.97 MiB | 1.54 B | Metal,BLAS | 16 |
pp512 | 5765.41 ± 20.55 |
# | qwen2 1.5B Q4_0 | 885.97 MiB | 1.54 B | Metal,BLAS | 16 |
tg128 | 197.71 ± 0.81 |
#
# build: 3e0ba0e60 (4229)
```

11 llama-run

A comprehensive example for running llama.cpp models. Useful for inferencing. Used with RamaLama ^3.

```
/*
杨小兵-2024-12-16

1、运行“llama.cpp”模型的综合示例，适用于推理，与 RamaLama 一起使用
2、`llama-run` 是一个用于运行 `llama.cpp` 模型的命令行工具，通常用于模型推理（即使用已经训练好的模型来进行实际的预测或任务处理）。它提供了一个综合示例，帮助用户轻松地启动并运行基于 LLaMA 模型的推理过程。

*/
```

- Run a model with a specific prompt (by default it's pulled from Ollama registry)

```
llama-run granite-code
```

12 llama-simple

A minimal example for implementing apps with `llama.cpp`. Useful for developers.

```
/*  
杨小兵-2024-12-16  
  
1、使用llama.cpp项目实现应用程序的一个最小的例子  
2、这个例子对开发者很有用  
*/
```

- ► Basic text completion

```
llama-simple -m model.gguf  
  
# Hello my name is Kaitlyn and I am a 16 year old girl. I am a junior in  
high school and I am currently taking a class called "The Art of
```

13 Contributing

- Contributors can open PRs
- Collaborators can push to branches in the `llama.cpp` repo and merge PRs into the `master` branch
- Collaborators will be invited based on contributions
- Any help with managing issues, PRs and projects is very appreciated!
- See [good first issues](#) for tasks suitable for first contributions
- Read the [CONTRIBUTING.md](#) for more information
- Make sure to read this: [Inference at the edge](#)
- A bit of backstory for those who are interested: [Changelog podcast](#)

```
/*  
杨小兵-2024-12-16  
  
1、贡献者可以开一个PRs  
2、协作者可以将branches推送到llama.cpp项目仓库中，并且可以将PRs合并到master分支中  
3、对于管理问题、PRs、项目的任何帮助将会十分感谢  
4、请参阅[good first issues]以了解适合首次贡献的任务  
5、读取CONTRIBUTING.md文件内容获取更多的信息  
6、确保将[Inference at the edge]内容了解了  
    6.1 边缘inference：  
7、给那些感兴趣的人讲一些背景故事  
*/
```

14 Other documentation

- [main \(cli\)](#)
- [server](#)
- [GBNF grammars](#)

```
/*  
杨小兵-2024-12-16  
  
1、程序示例：llama-cli  
2、程序示例：llama-server  
3、GBNF grammars  
*/
```

Development documentation

- [How to build](#)
- [Running on Docker](#)
- [Build on Android](#)
- [Performance troubleshooting](#)
- [GGML tips & tricks](#)

```
/*  
杨小兵-2024-12-16  
  
1、在PC上构建项目相关文档  
2、在docker上部署项目相关文档  
3、在Android上构建项目相关文档  
4、项目性能故障排除相关文档  
5、GGML库使用的技巧相关文档  
*/
```

Seminal papers and background on the models

If your issue is with model generation quality, then please at least scan the following links and papers to understand the limitations of LLaMA models. This is especially important when choosing an appropriate model size and appreciating both the significant and subtle differences between LLaMA models and ChatGPT:

- LLaMA:
 - [Introducing LLaMA: A foundational, 65-billion-parameter large language model](#)
 - [LLaMA: Open and Efficient Foundation Language Models](#)
- GPT-3
 - [Language Models are Few-Shot Learners](#)
- GPT-3.5 / InstructGPT / ChatGPT:
 - [Aligning language models to follow instructions](#)
 - [Training language models to follow instructions with human feedback](#)

```
/*  
杨小兵-2024-12-16  
  
1、开创性的论文和模型背景
```

2、如果您的问题与模型生成质量有关，请至少浏览以下链接和论文，以了解 LLaMA 模型的局限性。在选择合适的模型大小并理解 LLaMA 模型与 ChatGPT 之间的显著和细微差异时，这一点尤为重要

*/

References